ZeroTracer: In-Band eBPF-Based Trace Generator With Zero Instrumentation for Microservice Systems

Wanqi Yang, Pengfei Chen, Kai Liu, and Huxing Zhang

***Abstract*—Microservice enables agility in modern cloud-native applications but introduces challenges in fault troubleshooting due to its complex service coordination and cooperation. To tackle these challenges, distributed tracing has emerged for end-to-end request tracing and system understanding. However, existing tracing solutions often suffer from code instrumentation, trace loss and inaccuracy. To overcome these limitations, we introduce ZeroTracer, an in-kernel online distributed tracing system equipped with an eBPF-based (extended Berkeley Packet Filter) trace generator. ZeroTracer tailors for tracking HTTP requests due to its popularity in microservice systems. In our evaluations, ZeroTracer achieves remarkable trace accuracy (i.e., over 91%) and maintains stable performance under different workload concurrency. Moreover, ZeroTracer outperforms other non-invasive approaches which fail to reconcile accurate request causality. Notably, ZeroTracer effectively tracks end-to-end requests in multi-threaded microservice applications, which is absent in existing invasive distributed tracing systems with third-party library instrumentation. Moreover, ZeroTracer introduces a negligible overhead, with latency increasing by only 0.5% –1.2% and a modest 3% –5.8% increase in CPU and memory consumption.**

***Index Terms*—Distributed system, microservice, distributed tracing, eBPF, observability.**

I. Introduction

MODERN large-scaled distributed systems often adopt a microservice architecture to enhance scalability and reliability [1], [2], [3], [4], [5], [6], [7], [8]. However, managing such complex systems presents significant challenges. When a failure occurs, the complex interactions between services along with anomaly propagation complicate troubleshooting. To expedite troubleshooting, distributed tracing systems [9], [10], [11], [12], [13], [14], [15], [16], [17] are proposed and are now widely used in production environments. The latency of each service operation in end-to-end requests collected by distributed tracing systems significantly enhances the efficiency of troubleshooting and resolution for microservice applications, particularly in scenarios where anomaly propagation occurs [11], [18].

Distributed tracing systems are typically implemented in an *invasive* manner to track end-to-end requests in large-scaled microservice applications, like Zipkin [12], Jaeger [13], OpenTelemetry [14], OpenTracing [15], X-Trace [10] and Dapper [11], but suffer from high maintenance cost. Since a request may traverse numerous services deployed on different physical machines and geographical locations, and coded in various high-level programming languages, these invasive approaches require developers to weave tracing codes directly into the microservice applications or third-party libraries to automatically generate traces when applications handle requests. Nevertheless, code instrumentation in applications or third-party libraries lacks support for hot updates thus requiring high maintenance costs. Version updates can lead to incompatibilities, introducing potential security risks, program bugs, or even fatal errors.

Non-invasive distributed tracing systems are being studied as an alternative approach to reduce the maintenance cost introduced by the invasive ones, but have shortcomings in trace loss and inaccuracy. Existing non-invasive distributed tracing systems [19], [20] implement aggregation algorithms based on their collected network metrics and metadata. Although these non-invasive implementations eliminate code instrumentation, they often fail to capture accurate request causality in complex microservice applications. Moreover, their adopted offline analysis is not real-time and cannot support downstream service orchestration, such as gray release and request routing.

As outlined above, the implementation of a practical non-invasive distributed tracing system capable of effectively tracking end-to-end requests for microservice systems encounters several challenges as follows.

- *Compatible with various high-level languages:* Modern microservice systems are developed in various high-level languages, including Java, Python, and Golang. Ensuring compatibility with these commonly used languages while minimizing maintenance costs poses a significant challenge in tracking end-to-end requests.
- *Zero instrumentation with accurate traces:* In large-scale microservice systems, refactoring complex request causality without any code instrumentation presents a formidable challenge, particularly in containerized cloud-native environments. Moreover, microservice systems utilize either single-threaded or multi-threaded models, including thread pools or coroutines, further complicating non-invasive request tracing.

Received 22 April 2024; revised 5 April 2025; accepted 11 May 2025. Date of publication 19 May 2025; date of current version 9 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62272495 and in part by the Guangdong Basic and Applied Basic Research Foundation Grant 2023B1515020054. This work was supported in part by Alibaba Project under Grant 20925056, for which we are deeply grateful. Recommended for acceptance by S. Al-Kiswany. *(Corresponding author: Pengfei Chen.)*

Wanqi Yang and Pengfei Chen are with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510275, China (e-mail: yangwq23@mail2.sysu.edu.cn; chenpf7@mail.sysu.edu.cn).

Kai Liu and Huxing Zhang are with Alibaba Group, Hangzhou 311121, China (e-mail: qianlu.lk@alibaba-inc.com; huxing.zhx@alibaba-inc.com).

Digital Object Identifier 10.1109/TPDS.2025.3571934

1045-9219 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

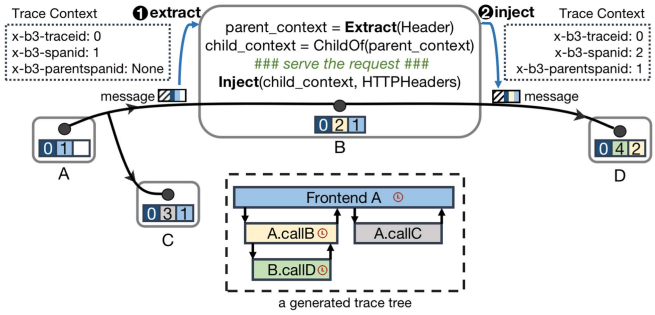


Fig. 1. An example to show the trace context propagation across the network and through services for generating a trace.

- *Online request tracing:* Online tracing offers real-time visibility into how requests flow through a distributed system, facilitating quick issue identification and resolution. However, tracking requests online without any code instrumentation poses considerable difficulty.

To address these challenges, we introduce ZeroTracer, an in-kernel non-invasive distributed tracing system leveraging eBPF (extended Berkeley Packet Filter) [21], [22]. With the widespread use of HTTP (HyperText Transfer Protocol) in modern microservices [23], [24], ZeroTracer is designed to realize real-time end-to-end HTTP request tracing for microservice applications implemented in various high-level languages. By intercepting requests and responses, injecting custom trace contexts (e.g., trace ID) as tags into the messages, and extracting those contexts from messages, ZeroTracer propagates trace contexts within the kernel with its automated discovery of parent-child relationships among threads or coroutines. Consequently, ZeroTracer can automatically track end-to-end requests in the kernel without user awareness. Our evaluation demonstrates that ZeroTracer effectively tracks over 91% of end-to-end requests for both single-threaded and multi-threaded microservice applications, even under high concurrency. Furthermore, ZeroTracer exhibits minimal impact on service performance, with just a 0.5% –1.2% increase in service latency and additional consumption of 3% –5.8% in CPU and memory resources. Moreover, we have successfully integrated ZeroTracer into a real-world cloud tracing console, demonstrating its practical applicability and effectiveness.

Contributions of our work can be summarized as follows.

- We implemented a generic distributed tracing system in the kernel to facilitate request tracing for microservice applications developed in diverse high-level languages, reducing the maintenance cost associated with frequent updates to third-party tracing libraries or application source codes.
- We study how microservice applications serve a user request, and implement in-kernel automated trace context propagation through single-threaded and multi-threaded containerized services with our automated discovered parent-child relationships among threads and coroutines.
- We utilize eBPF to implement an in-kernel trace generator to support online request tracing with zero instrumentation into microservice applications. This capability provides real-time observability of microservice systems, distinguishing our system from existing non-invasive tracing systems that typically use eBPF as a monitor or span generator.
- We propose a non-invasive in-band distributed tracing system, named ZeroTracer. We have conducted extensive experiments and deployed ZeroTracer in a real-world environment. Experiment results show that ZeroTracer can achieve impressive tracing accuracy over 91% with negligible overhead, 0.5% –1.2% increases in latency and an additional 3% –5.8% in CPU and memory consumption.

II. Background and Motivations

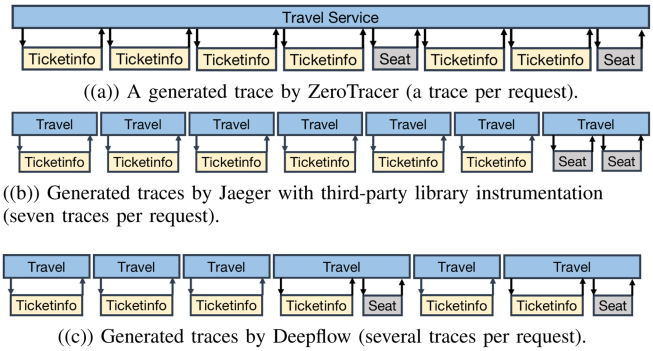
A. Request Tracing With Zero Instrumentation

1) Distributed Tracing System: Distributed tracing systems [9], [10], [11] enable tracking end-to-end requests across large-scaled microservice systems. They provide insights into the latency contributed by each operation within an end-to-end request and illustrate the request causality, which can be displayed in visual representations such as trace trees. Analyzing these trace trees helps developers pinpoint bottlenecks and troubleshoot performance issues associated with anomalous end-to-end requests. As the microservice architecture gains popularity, distributed tracing systems have become indispensable tools for effective system management.

Fig. 1 illustrates how a distributed tracing system works to help with fault troubleshooting. Traces are generated by trace context propagation alongside the end-to-end requests. For example, service B extracts the parent trace context from the request coming from A and injects child trace context into its request to D, to facilitate trace context propagation and associate these two requests with their causality. Each service operation in the end-to-end request generates a span. By aggregating the generated spans with the same trace ID (0 in this case) from all services, the distributed tracing system can visualize an end-to-end request in a trace tree as shown in the dashed box. When a failure on request latency happens in service D, it propagates to service B and then to front-end A. We can traverse the trace tree from the root span (span 1) to its children spans (span 2 and then span 4), comparing the latency of each span along the way to diagnose the failure. This example highlights the significance of propagating trace contexts for effective request tracing and fault troubleshooting.

2) Drawbacks of Existing Implementations: Existing distributed tracing systems are commonly implemented in an **invasive** manner [11], [12], [13], [14], [15], requiring code instrumentation in applications or third-party libraries to generate traces manually or automatically. While these systems can track requests to enhance troubleshooting efficiency, they are accompanied by certain drawbacks: ①*High maintenance cost:* Microservice applications are developed across a spectrum of high-level programming languages. As a result, traditional distributed tracing systems often necessitate the provision of instrumented third-party libraries in multiple high-level languages to facilitate end-to-end request tracing. This process of instrumenting libraries is notably time-consuming, especially when

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.



((a)) A generated trace by ZeroTracer (a trace per request).

((b)) Generated traces by Jaeger with third-party library instrumentation (seven traces per request).

((c)) Generated traces by Deepflow (several traces per request).

Fig. 2. An example of generated traces by ZeroTracer, Deepflow, and Jaeger with third-party library instrumentation when posting a request to the multi-threaded Travel service of the Trainticket application.

accommodating emerging programming languages. Moreover, the incessant updates to third-party libraries and applications exacerbate the challenge of maintaining instrumented tracing codes. For example, OpenTelemetry necessitates instrumenting at least 8 lines of codes for manual trace context propagation for each span in a C++ application [25]. If a C++ program generates a trace with one hundred spans, it requires at least eight hundred lines of instrumented codes, all of which need to be maintained. ②*Low accuracy with third-party library instrumentation:* Trace accuracy refers to its ability to precisely reflect request causality as defined in application source codes. Existing invasive distributed tracing systems can produce accurate traces by manually propagating trace contexts through requests (Fig. 1), but fall short in trace accuracy when applications handle requests across multiple threads or coroutines with third-party library instrumentation. For instance, as illustrated in Fig. 2(b), Jaeger with third-party library instrumentation fragments an end-to-end request served by the multi-threaded Trainticket application (see Section V).

Given the drawbacks of current invasive distributed tracing systems, developers are exploring the implementation of general **non-invasive** distributed tracing systems. These systems operate independently of high-level languages, aiming to reduce the maintenance costs on tracing code instrumentation. However, these non-invasive systems also exhibit some shortcomings. ①*Inaccuracy in inferred request causality:* Existing non-invasive distributed tracing systems [19], [20] commonly rely on algorithms to infer request causality without requiring any code instrumentation in microservice applications. However, their refactored request causality may be inaccurate due to their lack of insights into application implementations. For instance, Deepflow [20] fails to track certain end-to-end requests under specific workloads or within complex microservice applications due to limitations in its aggregation algorithm (see Sections V-C and V-D). Fig. 2(c) illustrates one of the inaccurate traces generated by Deepflow for the multi-threaded Trainticket application. ②*Lack of support for online tracing:* Existing non-invasive distributed tracing systems rely on offline analysis and cannot support real-time request tracing. Moreover, the significant volume of data gathered within a large-scale microservice system can adversely impact the performance of these offline analysis algorithms, leading to increased runtime overheads.

TABLE I
FEATURE COMPARISON BETWEEN ZEROTRACER AND EXISTING DISTRIBUTED TRACING SYSTEMS

Methods	Accuracy	Code Instrumentation	Context Propagation	Trace Generation
Application Instrumentation	Good	✔	In-Service	Online
Libaray Instrumentation	Fair	✔	In-Service	Online
Odigos	Fair	✔	In-Service, In-Kernel	Online
Deepflow	Fair	✘	✘	Offline
ZeroTracer	**Good**	✘	**In-Kernel**	**Online**

3) Goals of ZeroTracer: The pros and cons of existing distributed tracing systems analyzed above put forward the goals for ZeroTracer summarized as follows. A detailed comparison between ZeroTracer and existing distributed tracing systems is presented in Table I.

- *Goal 1:* Zero instrumentation. ZeroTracer must track end-to-end requests with zero instrumentation into applications or third-party libraries, thereby minimizing maintenance costs.
- *Goal 2:* High-level language support. ZeroTracer should integrate with commonly used high-level languages in microservice systems, such as C, Python, and Java, etc.
- *Goal 3:* High accuracy. ZeroTracer should ensure accurate request tracing through microservice applications, even if they are implemented in multi-threaded or coroutine-based architectures.
- *Goal 4:* Low overhead. ZeroTracer should track requests with a low overhead.
- *Goal 5:* Online tracing. ZeroTracer should provide real-time observability of microservice systems, enabling rapid identification and resolution of performance issues.

Note: As HTTP is widely used in 54% -87.5% of existing microservice systems, provided by the top-2 Chinese cloud platform provider, ZeroTracer aims to implement HTTP request tracing.

B. Why eBPF?

BPF (Berkeley Packet Filter) [26] was designed for packet filtering and later extended as eBPF (extended BPF) [21], [22] to enable user-defined event-triggered programs to execute in the Linux kernel. eBPF provides a set of network hooks for eBPF programs to be attached, thereby enhancing network capabilities [27], [28], [29], [30], [31], [32], [33].

ZeroTracer utilizes eBPF rather than LKM (Linux Kernel Module) [34] to execute customized lightweight programs within the Linux kernel, as eBPF has predefined hooks on Linux network kernel functions and owns strict security verification. In contrast to existing non-invasive tracing systems that employ eBPF solely as a monitor or span generator, ZeroTracer adopts eBPF as a **trace generator** for online in-band end-to-end request tracing. By attaching eBPF programs to kernel and user functions as well as sockets, ZeroTracer propagates trace

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

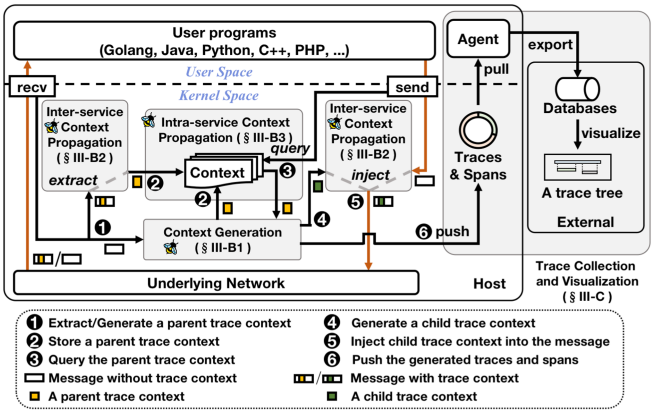


Fig. 3. An overview of the workflow of ZeroTracer.

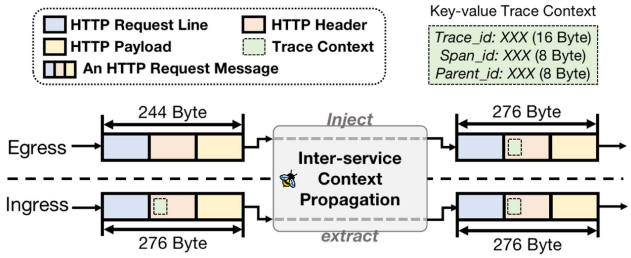


Fig. 4. Trace context injection and extraction.

contexts through requests in the kernel. This approach enables ZeroTracer to achieve end-to-end request tracing without any instrumentation into microservice applications, thereby supporting their implementations across various high-level languages (**Goal 1, 2**). Notably, ZeroTracer predominantly utilizes the *hash* eBPF Maps, which serve as efficient hash tables, for temporary key-value datastores. Benefiting from existing eBPF wrappers [35], [36], ZeroTrace is implemented by libbpf for its low overhead (**Goal 4**).

III. System Design

A. Overview

In this paper, we propose a non-invasive eBPF-based distributed tracing framework, namely ZeroTracer. Fig. 3 shows the workflow of ZeroTracer, where red lines represent the direction of message transmission and black lines represent the data flow of ZeroTracer. ZeroTracer first attaches eBPF programs to the kernel functions of each host on message transferring, such as `sendmsg` and `recvmsg`. When a service receives a request from its downstream, the eBPF program on the receiving side parses and extracts a trace context from the received message if it has, or generates a trace context if not (step ❶). The eBPF program then stores the context into an eBPF map as a parent trace context (step ❷). When the service sends a request to its upstream, the eBPF program on the sending side queries the parent trace context (step ❸), then generates its child trace context (step ❹), and injects the child context into the sending message (step ❺). If a request is finished, the eBPF programs will push the generated traces and spans into a ringbuffer, an eBPF map (step ❻). An agent is deployed on each host to facilitate access to the generated traces. This agent retrieves traces from the ringbuffer and exports them to databases following the OpenTelemetry standard. Users can then access and visualize these traces through a customized user interface tailored to their needs.

B. Trace Generation

In order to track end-to-end requests and generate traces, ZeroTracer first generates a unique trace context for each operation within the end-to-end request. Subsequently, it propagates this context alongside messages through the network, and then through the services themselves.

1) Context Generation: Inspired by existing distributed tracing systems [12], [14], [15], ZeroTracer divides a request into two separate operations: the client requests the server and the server responds to the client. Each of these two operations can be viewed as a span in an end-to-end request. The latency of these two spans is measured by timestamps recorded when messages are sent and received. The span generated by the entry service when it is serving the entry request is regarded as the root span of a trace. The other operations to serve that entry request are all considered as children spans of that root span.

To make end-to-end requests distinguishable, ZeroTracer assigns a unique identifier known as the trace ID to each end-to-end request. Similarly, individual operations in an end-to-end request are distinguished by span IDs. ZeroTracer generates these unique trace contexts according to established rules used in production environments, adhering to the widely recognized OpenTelemetry standard. Trace ID is a 16-byte value including a 32-bit host IP address, a 52-bit timestamp indicating when the end-to-end request entries, a 16-bit sequence number representing the order of requests to the entry service, a 4-bit debugging tag used by developers, and a 24-bit running process ID. Meanwhile, span ID is an 8-byte value comprising the 16-bit IP addresses of the client and server, as well as a 32-bit sequence number denoting the order of the span within the entire trace.

2) Inter-Service Context Propagation: After generating a unique trace context for each operation in an end-to-end request, ZeroTracer has to propagate these contexts between services. By analyzing the format of HTTP requests, ZeroTracer utilizes the key-value header of HTTP requests as the optimal location for carrying trace contexts, thereby facilitating online request tracing (**Goal 5**). Along with HTTP requests, trace contexts can easily travel across the network, transferred between two services. Moreover, since the injection of trace contexts does not alter the payload, it ensures that the normal processing of original requests by microservice applications remains undisturbed.

As depicted in Fig. 4, when a service sends an HTTP request to its upstream, ZeroTracer injects a trace context as custom key-value pairs into the header of the HTTP request before transmission. This injection increases the message length by 32 bytes, from 244 bytes to 276 bytes. Upon receiving an HTTP request from the downstream containing our customized trace context, ZeroTracer intercepts the request and extracts the context without impacting the message length, which remains at 276 bytes.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

3) Intra-Service Context Propagation: To achieve accurate end-to-end request tracing, ZeroTracer should effectively propagate the trace context through each service of a microservice application from its downstream request to its upstream request (**Goal 3**).

While existing invasive distributed tracing systems rely on instrumenting tracing codes in applications or third-party libraries to implement context propagation, ZeroTracer faces the challenge of achieving trace context propagation in the kernel without any code instrumentation, since ZeroTracer has limited knowledge of how a running application serves its user request. To address this challenge, we analyze the workflow of services in microservice applications to understand how they serve a user request. As one service is scheduled on one user process to execute by the operating system, we observe that the scheduling of a running service on a user process remains consistent. Leveraging this insight, ZeroTracer can monitor the user thread creation and execution in the kernel to understand the execution of user threads as well as their parent-child relationships within the service process. As a result, ZeroTracer gains the capability to *propagate trace context with the relationships among threads* and without any code instrumentation in the kernel, regardless of the various high-level language implementations (**Goals 1, 2, and 3**).

Services primarily serve requests in two modes.

- Single-threaded mode: Single-threaded services serve a request within a single thread. If the serving thread requires requesting its upstreams, it will send the request and wait for the response within the same thread.
- Multi-threaded mode: Multi-threaded services always serve a request with multiple collaborating threads or even coroutines. When the serving thread requests its upstream for services, the service creates or schedules one of its idle threads to request the upstream and continues to serve the user request with no wait.

Based on these two implementations, we discuss how ZeroTracer propagates trace contexts through services as follows.

For **single-threaded services**, ZeroTracer propagates trace contexts using the unique identifier of each thread, known as the thread ID. Since a thread serves a request synchronously and individually (see Fig. 5(a)), ZeroTracer first stores the extracted trace context with the thread ID as a key when receiving a request from the downstream. Subsequently, when the service forwards a request to its upstream to fulfill the downstream request, ZeroTracer retrieves the trace context associated with the same thread ID as the parent trace context. ZeroTracer then generates a child trace context and injects it into the outgoing upstream request. With the same thread ID, trace contexts can be propagated through the single-threaded services to achieve end-to-end request tracing in single-threaded services.

For **multi-threaded services**, where tasks are parallelized to enhance service performance, ZeroTracer encounters a more complex scenario. In the simplest scenario of a multi-threaded service, when a thread serves a downstream request, it forks or clones a new thread to request its upstream (see Fig. 5(b)). By monitoring the thread creation, ZeroTracer establishes the parent-child relationships among threads. Leveraging these relationships, ZeroTracer can implement context propagation for such multi-threaded services. ZeroTracer first stores the received trace context as the parent trace context with the parent thread ID as the key. When the child thread sends a message to the upstream, ZeroTracer queries its parent thread ID, retrieves the parent trace context, generates a child trace context, and injects it into the the sending message.

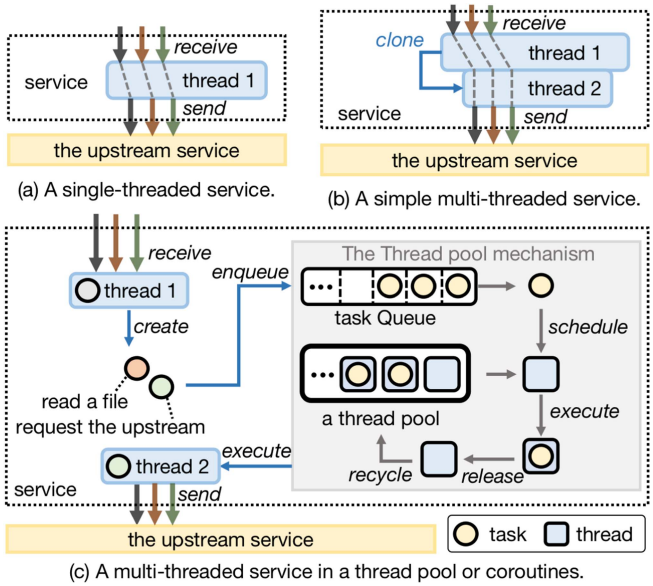


Fig. 5. Implementations of microservices on how they serve a request.

In more complex scenarios, multi-threaded services prefer to utilize **thread pools**, even **coroutines** to enhance performance by mitigating the system resource overhead associated with frequent thread creation and destruction. Fig. 5(c) displays how a thread-pool service serves requests. When serving a downstream request, the service creates some collaborative tasks or coroutines to execute various tasks, such as requesting upstream services or reading a file. These created tasks or coroutines are then put into a queue, waiting to be scheduled on an idle thread in the thread pool to execute. If a thread finishes its running task, it will be recycled back into the thread pool and remain idle until another task is scheduled on it. Although tasks and coroutines offer fine-grained resource allocation and scheduling in user space to optimize service performance, propagating trace context through such a complex thread-pool service presents a formidable challenge in the kernel.

Fig. 6 shows the workflow of ZeroTracer to propagate trace contexts through thread-pool services. In thread-pool environments, where different tasks can be scheduled on the same thread for execution, the conventional approach of propagating context using thread IDs, as employed in simple multi-threaded scenarios, proves inadequate. To accurately propagate trace contexts in thread-pool services, ZeroTracer has to gain insights into the scheduling of tasks or coroutines on threads. By attaching an eBPF program on task creation events, ZeroTracer captures the parent-child relationships among tasks. The scheduling and

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

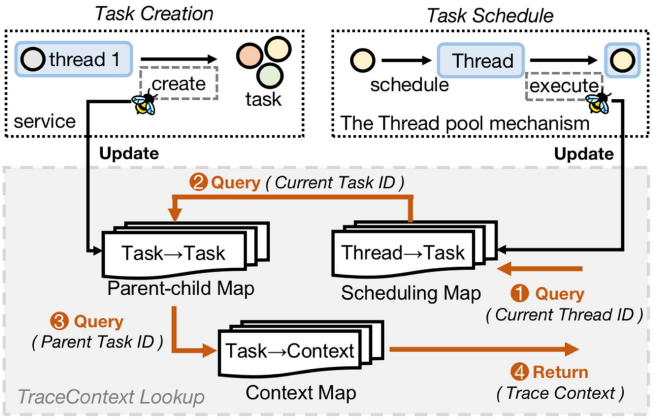


Fig. 6. The solution of ZeroTracer to propagate trace contexts through thread-pool services.

TABLE II
IMPORTANT MOUNTING HOOKS OF eBPF PROGRAMS IN ZEROTRACER

Type	Hook	Functionality
sock_ops	-	filter target sockets
sk_msg	-	inject trace contexts into messages
kprobe	sock_recvmsg	record the addresses of received buffers
	sock_sendmsg	recognize the response messages
kretprobe	sock_recvmsg	extract trace contexts from messages
	alloc_pid	find parent-child relationships of threads
uprobe	execute	acquire coroutine scheduling information
	newproc1	find parent-child relationships of coroutines
	ServeHTTP	mark the serving coroutines
	writeloop	associate write coroutine and its connection
	readloop	associate read coroutine and its connection
	roundTrip	update parent-child relationships of coroutines for keep-alive connections
uretprobe	newproc1	find parent-child relationships of coroutines

execution of tasks on threads are monitored to gather scheduling details. When a thread receives a downstream request alongside a trace context, ZeroTracer stores this context as the parent trace context, associating it with a unique identifier of the executing task as the key, known as the task ID. Subsequently, when a thread is requesting an upstream service, ZeroTracer first queries whether the executing task owns a trace context. If so, this context will be treated as the parent trace context. Otherwise, ZeroTracer will query for the parent trace context associated with the parent task of the executing task. Then ZeroTracer generates a child trace context and injects it into the sending message. By leveraging fine-grained scheduling information and parent-child relationships among tasks, ZeroTracer effectively propagates context through thread-pool services.

C. Trace Collection and Visualization

Once trace contexts have been generated and propagated across the network and through services, ZeroTracer constructs a unique trace for each finished end-to-end request. Traces along with their spans are pushed into a ringbuffer, an eBPF map, and subsequently retrieved by an agent deployed on the host. For long-term storage, the agent exports these traces to local or external databases. Traces in the databases can be accessed and visualized with a dashboard to better help developers debug anomalous end-to-end requests and troubleshoot faults. To be compatible with the existing visualization user interface, our agent transforms the generated traces into the standard format of OpenTelemetry before exporting them into databases.

IV. IMPLEMENTATION

ZeroTracer is implemented in C/C++ with about 2000 lines for the in-kernel eBPF programs and about 1500 lines for the host agent. ZeroTracer utilizes six types of the eBPF programs to realize a non-invasive distributed tracing system: `sock_ops`, `sk_msg`, `kprobe`, `kretprobe`, `uprobe`, and `uretprobe`. Table II shows their functionalities in detail.

A. Context Generation

By parsing and matching messages in HTTP request format when a service sends a request to its upstream or receives a request from its downstream, ZeroTracer generates 8-byte span IDs and 16-byte trace IDs based on the rules mentioned in Section III-B1. In addition, thanks to the service management provided by Kubernetes [37], each service and its instances have their IP address. By message filtering with IP addresses, ZeroTracer can further reduce the time cost on message parsing and request tracing for non-target requests, thus improving the performance of end-to-end request tracing for microservice applications (**Goal 4**).

B. Inter-Service Context Propagation

For trace context propagation between services, the generated trace context must be injected into outgoing requests to upstreams and extracted from incoming requests from downstreams. Details on the implementation of inter-service context propagation are illustrated in Fig. 7. To achieve this seamlessly, ZeroTracer implements a `sk_msg` program to intercept outbound HTTP requests and inject customized trace contexts without user intervention. Benefiting from `bpf_msg_push_data`, an eBPF helper function, the `sk_msg` program first splits a message into two fragments in a scatter list, then injects our customized trace context as the first key-value data in the HTTP header after the request line in the first fragment (Line 9-10). After context injection, these two fragments are merged back into a request to be sent. When the service receives the downstream request alongside a trace context, our `kprobe` and `kretprobe` programs hooked on the kernel function `sock_recvmsg` help parse that downstream request and extract our injected trace context (Line 1-3), then store the context with the thread ID or task ID for intra-service context propagation (Line 4-7).

In our implementation, the request line can be recognized by the hexadecimal ending symbol `0x0d0a` of each line in an HTTP request (Line 4-8). By injecting trace contexts in the first place of the header after the request line, ZeroTracer

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

```
Program sk_msg program(s)
Input: msg - A Sending Message                    bpf_tail_call
1  tid = getThreadID()
2  ptcontext = lookupContextByThreadID(tid)
3  ctcontext = generateChildTraceContext(ptcontext)
   // find the end of the request line
4  first_end_pos = 0
5  while i = 1 to Len(msg) do
6     if msg[i] == 0x0d0a then
7        first_end_pos = i
8        break
   // extend the length of msg and inject trace context
9  bpf_msg_push_data(msg, SizeOf(ctcontext))
10 injectTraceContext(msg, first_end_pos, ctcontext)

Program (kretprobe)sock_recvmsg program
Input: msg - A Receiving Message
1  tid = GetThreadID()
2  task_id = lookupRunningTaskID(tid)
3  tcontext = ExtractTraceContextFromMsg(msg)
4  if task_id then
5     storeTraceContexWithThreadID(tcontext, tid)
6  else
7     storeTraceContextWithTaskID(tcontext, task_id)
```

Fig. 7. Implementation of the sk_msg program and the kretprobe program hooked on the sock_recvmsg function for inter-service trace context propagation.

decreases the number of operating instructions needed to locate other key-value pairs in the header for context injection, as eBPF has an upper limit on the number of instructions for each eBPF program due to its security considerations. We choose to implement trace context injection at the socket layer rather than the transport layer and lower to avoid the impact of the transport mechanisms on request tracing, such as packet subcontracting and retransmission.

In practice, the length of the request line in an HTTP request is variable, so eBPF programs have to traverse the HTTP request messages to figure out the first ending symbol `0x0d0a` (Line 4-8) and inject the generated contexts (Line 9-10). However, the traversal for the longer request line in one eBPF program makes it out of its limitation on the number of running instructions. To meet this problem, ZeroTracer employs the tail call mechanism of eBPF to increase the number of available eBPF instructions, as highlighted by the orange boxes in Fig. 7. ZeroTracer divides the process of injecting a trace context into three parts: generating the context to be injected (Line 1-3), finding the injection location (Line 4-8), and injecting the context (Line 9-10). Each is implemented by an eBPF program. By calling the `bpf_tail_call` helper function, ZeroTracer can easily utilize these three eBPF programs to successfully inject trace contexts into requests and then propagate them across services.

C. Intra-Service Context Propagation

As analyzed in Section III-B3, propagating trace contexts through services requires implementing the `lookupContextByThreadID` function used in the sk_msg program (Line 2 in Fig. 7, with details in Fig. 9). To achieve this,

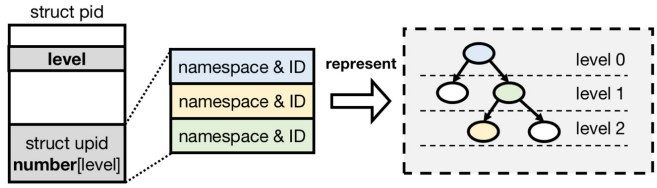


Fig. 8. Thread ID assignment in Linux. Each thread ID in the child namespace has a unique ID in each of its parent namespaces.

ZeroTracer needs to know about the parent-child relationships among threads, tasks, and coroutines, as well as the scheduling information of tasks and coroutines on threads.

In microservice systems, services are deployed in containers to achieve fast deployment. However, resource isolation in container namespaces [38] makes it a challenge to build up parent-child relationships among threads in the kernel. Threads of a containerized service execute in their container namespace and are assigned unique IDs of that namespace and so do in its parent namespaces, such as the root (host) namespace. *The IDs of a thread are different from namespaces.* Instead of deploying ZeroTracer in each container namespace, which consumes more system resources, ZeroTracer operates in the root namespace, while services execute in their respective child namespaces. This disparity in thread IDs introduces inconsistencies and inaccuracies in establishing parent-child relationships among threads when monitoring the kernel function `fork` and `clone` in the root namespace. Since different threads may utilize the same thread ID in their respective namespaces, retrieving thread IDs within child namespaces using these functions can lead ZeroTracer to the inaccuracy in establishing parent-child thread relationships. To meet this problem, we try to figure out how ZeroTracer can obtain the thread IDs of the containerized services in the root namespace. Fig. 8 shows the data structure of a thread in Linux, where a thread is regarded as a process to execute. Each thread records its namespace level in a variable called `level` and maintains its thread IDs for each namespace in an array named `number`. These IDs are recorded following the parent-child relationships of namespaces, starting from the parent namespace and ending with the current namespace. When a new thread is forked, it is allocated new IDs in all its located namespaces by calling the kernel function `alloc_pid`. The `kprobe` program on that function can help ZeroTracer to obtain the structure of a thread and parse its ID in the root namespace, thus ZeroTracer is able to *keep the consistency when establishing the parent-child relationships of threads* (used in Line 20, Fig. 9).

To facilitate intra-service context propagation, ZeroTracer utilizes `uprobe` and `uretprobe` programs attached to task scheduling and creation functions. These programs enable ZeroTracer to gather parent-child relationships among tasks, along with their scheduling details on threads (used in Line 1, Line 6, and Line 11 in Fig. 9). During context propagation, ZeroTracer initially stores the extracted parent trace context using the task ID as the key. If no running task is present, it changes to use the thread ID as the key. When a service sends requests to upstreams, ZeroTracer queries the parent trace context and

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

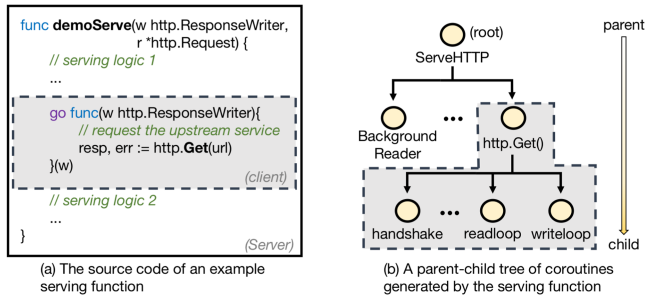
```
function LOOKUPCONTEXTBYTHREADID(tid)
Input: tid - Thread ID, L - Parent Level
Output: tcontext - Trace Context
 1  task_id = lookupRunningTaskID(tid)
 2  if task_id then
 3     tcontext = findTraceContextByTaskID(task_id)
 4     if tcontext then return tcontext
 5     else
 6        ptask = findParentTaskID(task_id)
 7        tcontext = findTraceContextByTaskID(ptask)
 8        level = L - 1
 9        while ptask and !tcontext and level > 0 do
10           task_id = ptask
11           ptask = findParentTaskID(task_id)
12           tcontext = findTraceContextByTaskID(ptask)
13           level - -
14        if tcontext then return tcontext
15        else return None
16  else
17     tcontext = findTraceContextByThreadID(tid)
18     if tcontext then return tcontext
19     else
20        parent_tid = findParentThreadID(tid)
21        tcontext = findTraceContextByThreadID(parent_tid)
22        if tcontext then return tcontext
23        else return None
```

Fig. 9. Implementation of the function on looking up the trace context by thread ID.

generates a corresponding child trace context for injection. The lookup procedure is shown in Fig. 9. If a thread scheduled for an upstream request has an associated task, ZeroTracer first searches for the trace context using the task ID (Line 3-4). If not found, it continues to traverse up the parent-child hierarchy until a valid trace context is found or the predefined level of hierarchy (L) is reached (Line 6-14). If the trace context of the L-parent task is still unrecognized, it is considered that no trace context needs to be propagated (Line 15). If the current thread is not performing a task, ZeroTracer retrieves the parent trace context using the current thread ID (Line 17-18), or if unavailable, the parent thread ID (Line 20-21). Once the trace context is found, it is regarded as the parent trace context and used to generate a child trace context to be injected into the request to the upstream to enable context propagation. Since upstream requests are typically parallelized at a 1-level parent-child thread relationship, ZeroTracer only searches for trace context at this level. Conversely, task hierarchies tend to be deeper, so ZeroTracer extends the search to a predefined maximum level (e.g., 10) for task-related contexts.

D. Golang Example of Intra-Service Context Propagation

To better explain which user functions ZeroTracer chooses to attach and how ZeroTracer propagates trace contexts in thread-pool services, we show our implementation in a Golang service as an example, which natively implements thread pools in its runtime. For Java and Python, unlike Golang, which allows eBPF programs to hook user functions using the function address, their packaged execution environments, such as the Java Virtual Machine (JVM) and the Python Virtual Machine (PVM), make it challenging for eBPF programs to retrieve and attach to dynamic



(a) The source code of an example serving function

(b) A parent-child tree of coroutines generated by the serving function

Fig. 10. An example of a serving function in Golang with its generated parent-child tree of coroutines.

function addresses. To overcome this, ZeroTracer can leverage bytecode instrumentation tools like javaagent [39] to hook key user functions, such as task scheduling and task creation. This enables ZeroTracer to understand scheduling relationships between tasks and threads, as well as parent-child relationships between tasks, facilitating intra-service context propagation.

As shown in Fig. 10, a Golang service using "net/http" library employs multiple coroutines to serve an HTTP request. Once receiving a request, the service creates a coroutine to execute the serving function shown in Fig. 10(a). The serving coroutine then creates child coroutines to handle various tasks such as reading request content, sending requests to upstreams, processing responses, and other operations. This hierarchy of coroutines forms a parent-child tree, illustrating the flow of operations within the service, as shown in Fig. 10(b). To propagate context through such a Golang service, ZeroTracer hooks `uprobe` and `uretprobe` programs on the user function `runtime.newproc1` to capture parent-child relationship among coroutines (used in Line 6 and 11 in Fig. 9) and on `runtime.execute` to obtain coroutine scheduling information on threads (used in Line 1, Fig. 9). In addition, the user function `HandlerFunc.ServeHTTP` is hooked by ZeroTracer to find the serving (root) coroutine of each parent-child tree of coroutines. When serving a downstream request, the root coroutine extracts a trace context from the downstream request and stores it with the root coroutine identifier (coroutine ID) as a key. When a child coroutine sends a request to the upstream, it first finds its belonging trace context with its coroutine ID. If it has, its belonging trace context is regarded as the parent trace context. If not, ZeroTracer queries for the parent trace context of its parent coroutines until the root coroutine, as illustrated in Fig. 9. Then ZeroTracer generates a child trace context of the founded parent trace context and injects it into the upstream request.

To keep the parent-child relationships of coroutines, we observe that keep-alive connections in the "net/http" library may cause wrong parent-child relationships of coroutines. As shown in Fig. 11, the write and read coroutines work with the keep-alive connections. However, a keep-alive connection can be acquired by different coroutines for requesting the same upstream. For example in Fig. 11, a keep-alive connection to an upstream can be acquired by coroutine 1, and then released by coroutine 1 and acquired by coroutine 2. ZeroTracer has to maintain accurate parent-child relationships of coroutines in this

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

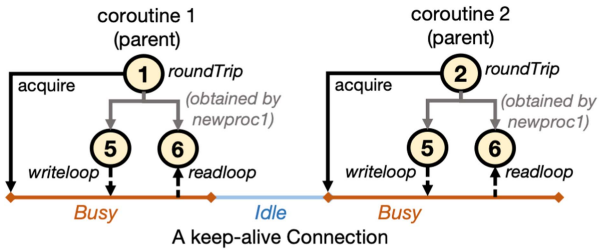


Fig. 11. Parent-child relationships of coroutines change on a keep-alive connection in a Golang service.

case to achieve trace context propagation (used in Line 6 and 11 in Fig. 9). In our implementation, ZeroTracer monitors the user functions `writeloop` and `readloop` to establish the owning relationship between the read and write coroutines and the keep-alive connections. The user function `roundTrip` then helps ZeroTracer know which serving coroutine acquires the keep-alive connection, thereby maintaining an accurate parent-child relationship between the serving coroutine and the read and write coroutine of that keep-alive connection. In Fig. 11, when the keep-alive connection is acquired by coroutine 1, the write and read coroutines 5 and 6 record their parent coroutine as 1. When coroutine 2 obtains the same connection, coroutine 5 and 6 change their parent coroutine to 2.

E. Traces Exportation

Our eBPF programs export traces and spans to a ringbuffer when the service receives the response from its upstream or send a response to its downstream. To ensure compatibility with existing tracing user interfaces, an agent deployed on each host reads the generated traces and spans from the ringbuffer. Subsequently, it exports them to external databases in compliance with the OpenTelemetry standard.

V. Evaluation

To evaluate the performance of ZeroTracer, we conduct a series of experiments to answer the following questions.

Q1: How does ZeroTracer perform in tracking end-to-end requests for applications developed in various high-level languages? (Section V-B)

Q2: What impact does the kernel function ”alloc_pid” have on improving the effectiveness of ZeroTracer in containerized microservice applications? (Section V-C)

Q3: How does ZeroTracer perform under different concurrency levels? (Section V-D)

Q4: How much is the overhead incurred by ZeroTracer on resource consumption and service performance? (Section V-E)

A. Experiment Settings

1) Environment: We have set up our test environment on a Kubernetes [37] platform with 5 physical machines, each having 16 GB RAM and eight 4-core 2.50 GHz Intel Xeon Platinum 8269 CPUs. The machines run on Centos-8-based OS and have Linux kernel version 5.10.134. To compile the eBPF programs, we have opted for *clang* and *LLVM* version 13. We have used Fortio [40] to generate user-defined workloads to our benchmarks, while the *sar* command has been utilized to obtain the CPU and memory usage of each machine.

2) Benchmark: For our experiments, we have deployed two open-source microservice application benchmarks as our testbed. Bookinfo [41] serves as an online bookstore, written in various programming languages, including Python, and Java. Trainticket [42] is a large-scale distributed train ticket booking system consisting of 41 microservices, written in Java. Each service of these two benchmarks owns one replica. Since these two benchmarks only provide single-threaded examples, we modify the source codes of the Productpage service of Bookinfo and the Travel service of Trainticket to build up simple multi-threaded applications.

Furthermore, we conducted benchmarks in both single-threaded and multi-threaded environments using commonly used high-level languages such as PHP, Java, Python, and Golang to evaluate the performance of ZeroTracer on tracking requests in various high-level languages. Each benchmark comprises three services: a frontend service, a middle service, and a backend service. In the Golang example, we simulated a multi-threaded environment using a thread-pool model.

3) Baselines: Jaeger [13] stands for a typical commonly-used invasive distributed tracing system, which adheres to the OpenTelemetry and OpenTracing standards. It directly instruments tracing codes into applications and third-party libraries, making it a highly accurate baseline for comparison. Deepflow [20] is the state-of-the-art non-invasive approach for tracking requests. It implements an aggregation algorithm based on the metadata captured by eBPF, where eBPF programs perform as a span generator. Comparing ZeroTracer with Deepflow provides insights into our performance and effectiveness.

4) Evaluation Metrics: To better evaluate the effectiveness of ZeroTracer, we adopt accuracy as our primary metric. Accuracy reflects the similarity between the traces generated by different tracing systems and the actual end-to-end request paths from the application source codes. The accuracy is obtained by calculating the proportion of accurate traces. The higher the accuracy, the more effective the tracing system. To measure the impact of ZeroTracer on service performance, we use two metrics: request latency and query-per-second (QPS). Request latency is used to show the impact of ZeroTracer on service response time. QPS is used to measure the throughput of services. In addition, we also measure the consumption of CPU and memory to help understand the resource consumption of ZeroTracer on the hosts.

B. (Q1) Multi-Language Supporting

To evaluate the performance of ZeroTracer in tracking end-to-end requests of microservice applications developed in multiple high-level languages, we use Fortio to generate a continuous and sequential workload of ten thousand requests for our single-threaded and multi-threaded benchmarks. The comparison results are presented in Fig. 12(a). PHP applications are tested only in single-threaded mode due to its widespread use. Conversely, Golang applications are natively implemented with thread pools in runtime.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

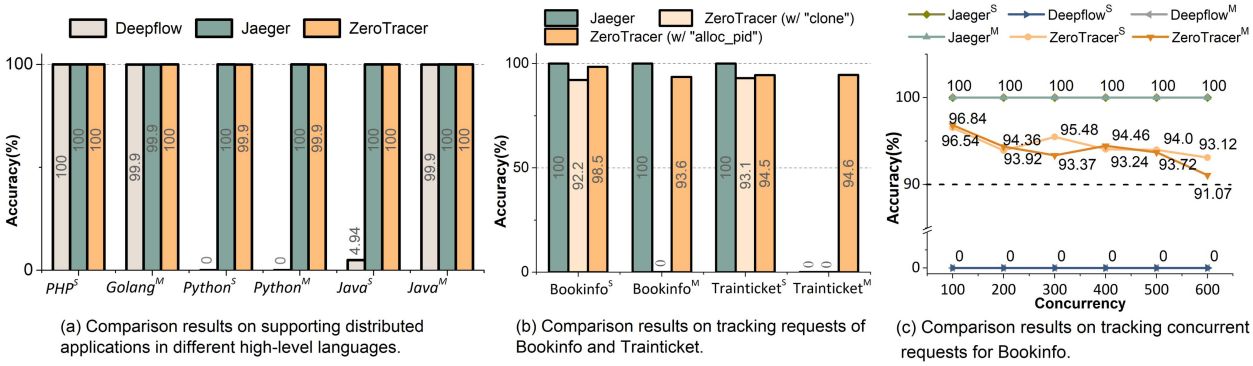


Fig. 12. Effectiveness comparison among Jaeger, Deepflow, and ZeroTracer on tracking end-to-end requests. In these figures,"S" represents the single-threaded services, while "M" represents the multi-threaded services.

As depicted in Fig. 12(a), Jaeger can always achieve perfect performance in tracking end-to-end requests by directly instrumenting tracing codes into microservice applications. In contrast, Deepflow relies on its monitored network metadata to analyze request causality, regardless of application implementations. While Deepflow successfully aggregates nearly all request causality for PHP, Golang, and multi-threaded Java microservice applications, it struggles to track end-to-end requests for Python and single-threaded Java applications. Analyzing the trace log collected from Deepflow, we discover that requests can not be correctly associated with the value of a tag named `syscall_trace_id`. In our experiments, over 75% of that value is zero, indicating a lack of causality relationships among requests and resulting in a failure to refactor end-to-end traces.

ZeroTracer enables tracking nearly 100% of end-to-end requests of multi-language applications, including those operating in multi-threaded mode. By monitoring parent-child relationships among threads and tasks, ZeroTracer identifies the ownership of requests to the threads or tasks or coroutines in the Linux kernel, thus propagating trace contexts through services without detailed invasive information of applications. Fig. 12(a) illustrates that ZeroTracer performs well in tracking end-to-end requests with zero instrumentation, regardless of the high-level languages employed in microservice applications.

C. (Q2) Effectiveness in Microservice Examples

To assess the performance of ZeroTracer on tracking end-to-end requests of containerized microservices, we select Bookinfo and Trainticket as our benchmarks. Fortio is set to generate a continuous and sequential workload of ten thousand requests for the single-threaded and multi-threaded Bookinfo and Trainticket. The comparison results on accuracy are shown in Fig. 12(b).

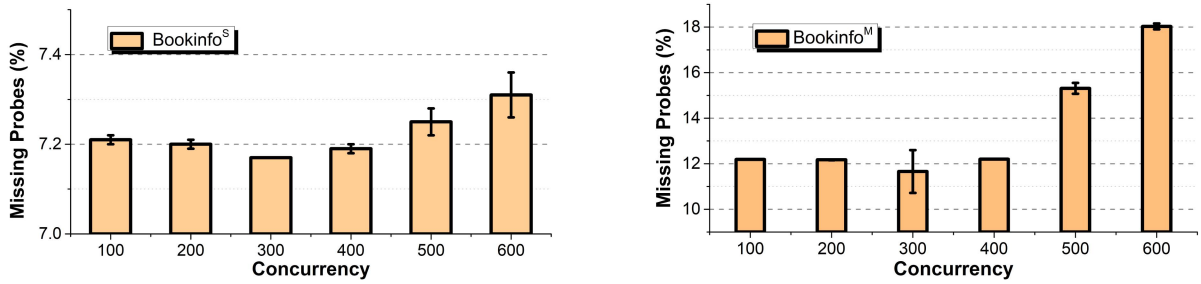
Jaeger keeps tracking end-to-end requests for Bookinfo accurately, thanks to its direct instrumentation into application codes. However, when it comes to TrainTicket, which utilizes instrumented third-party libraries, Jaeger performs well in tracking requests for the single-threaded Trainticket but encounters difficulties with the multi-threaded version. This discrepancy arises because instrumentations in third-party libraries for tracing tend to split a request processed by different threads into different traces.

Deepflow fails to refactor request causality with zero accuracy, whose results are not displayed in Fig. 12(b). The tag `syscall_trace_id` of trace logs collected by Deepflow owns zero value, resulting in the loss of children requests associated with its parent requests. Moreover, for an end-to-end request to the Travel service of Trainticket spanning 97 operations in a trace, the offline aggregation algorithm applied by Deepflow only refactors part of the entire request causality. This limitation arises due to the presence of nested requests and the extensive length of the request path, causing Deepflow to split the entire end-to-end requests into several traces.

ZeroTracer outperforms Deepflow when tracking end-to-end requests of containerized microservices. Since a container executes as a process in the host, ZeroTracer can effortlessly discern the parent-child relationship among threads and propagate trace context through the service. Consequently, ZeroTracer achieves accuracy exceeding 92% in tracking end-to-end requests, even for complex scenarios such as an end-to-end request comprising 97 spans in total for Trainticket. However, some inaccurate and incomplete traces may be generated when some services of Bookinfo and Trainticket fail to generate their spans properly. Observing the eBPF logs, we understand that the eBPF program fails to generate a span when the service sends a response to its downstream. This issue arises because eBPF programs start with the inserted instruction `int 3` into the hooked functions, but they may be interrupted and re-entered by other events invoking the same function [43], [44], [45]. Consequently, ZeroTracer may overlook a kprobe if an interrupt is already processing the same kprobe, leading to the loss of a span.

From Fig. 12(b), it is convincing that the eBPF program attached to the `alloc_pid` function significantly helps ZeroTracer establish parent-child relationships among threads for containerized service on the host, regardless of the namespace resource isolation. In scenarios where the function `clone` is monitored instead of `alloc_pid`, ZeroTracer effectively tracks end-to-end requests for the single-threaded Bookinfo and Trainticket application, where a thread processes a request individually. However, due to the namespace resource isolation, ZeroTracer encounters difficulties in identifying the parent-child relationships between threads residing in different namespaces, so ZeroTracer without monitoring `alloc_pid` fails to track end-to-end requests of multi-threaded Bookinfo and Trainticket.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.



((a)) Percentage of missing probes on single-threaded Bookinfo.

((b)) Percentage of missing probes in multi-threaded Bookinfo.

Fig. 13. Percentage of missing probes in ZeroTracer under different concurrent workloads for single-threaded and multi-threaded Bookinfo.

D. (Q3) Concurrency Impact on Effectiveness

To evaluate the performance of ZeroTracer under different concurrencies, we select Bookinfo as our benchmark since it can process up to six hundred concurrent requests simultaneously with a ten-second timeout. Fortio is configured to generate a total of ten thousand requests for both single-threaded and multi-threaded Bookinfo, with 100, 200, 300, 400, 500, and 600 concurrent users, respectively. Comparison results are presented in Fig. 12(c).

Jaeger keeps tracking all the end-to-end requests for either single-threaded or multi-threaded Bookinfo under different concurrent user workloads. With explicit instrumentation in source codes of Bookinfo, Jaeger can easily distinguish which downstream request causes an upstream request, thus tracking accurate request causality.

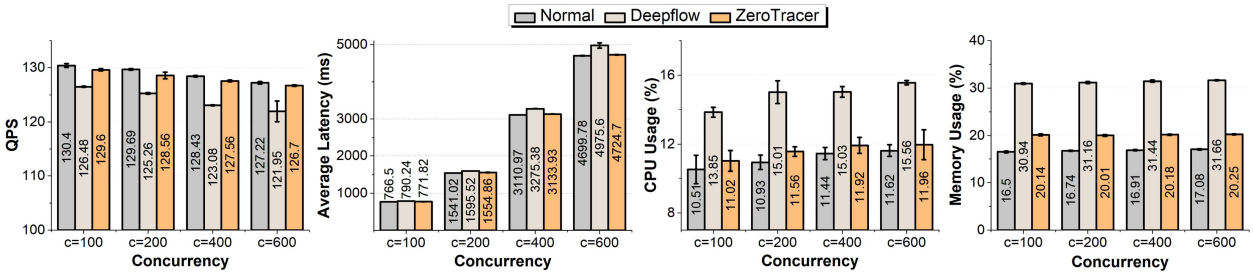
When tracking concurrent user requests, Deepflow proves ineffective in tracking any of the end-to-end requests for Bookinfo. Although Deepflow aggregates part of the end-to-end requests of Bookinfo, it falls short in aggregating the entire end-to-end requests due to its metadata-based algorithm. For instance, the Productpage service of Bookinfo requires requesting to the Detail service and the Reviews service to fulfill a user request. Deepflow prefers to treat these two requests within an end-to-end request as two separate traces rather than as part of the same trace. Deepflow assigns zero value to the tag `syscall_trace_id`, so it fails to properly associate the request from Productpage to Details with the request from Productpage to Reviews. Furthermore, Deepflow applies the policy that where multiple requests in the same transmission direction share the same trace ID, further muddles the causal relationship of requests. Consequently, Deepflow fails in aggregating accurate end-to-end requests.

ZeroTracer demonstrates remarkable accuracy, exceeding 91% when tracing concurrent end-to-end requests for both single-threaded and multi-threaded Bookinfo applications. Leveraging the parent-child relationships among threads for trace context propagation, ZeroTracer consistently delivers stable performance in request tracing. However, it falls short of achieving 100% accuracy as the eBPF programs hooked on the sending and receiving kernel functions can not perform properly and fail to generate spans, particularly under high concurrency scenarios. To identify the cause of the accuracy decline, we record the triggered count of each hooked eBPF program. We then compare the triggered count of eBPF programs for ten thousand requests under varying concurrency with the normal count of triggered eBPF programs for a single request, calculating the difference as the count of missing probes. The results are shown in Fig. 13. As the concurrency increases, the number of missing probes rises, particularly when the concurrency reaches 500 and 600, with missing probes ranging from 7.1% to 7.3% for single-threaded Bookinfo and from 12% to 18% for multi-threaded Bookinfo. The issue arises since eBPF programs rely on the inserted instruction `int 3` for entry, which can be interrupted and re-entered by other events invoking the same eBPF programs [43], [44], [45]. For example, an eBPF program may be triggered by a user-space application through system calls, but it can also be invoked by a hardware IRQ (Interrupt Request) or a soft IRQ, both of which have higher interrupt priority. If an eBPF program triggered by a user-space application is running, it can be interrupted and re-entered by a soft IRQ, leading to the loss of execution context. Consequently, ZeroTracer may miss the execution of a kprobe if a kprobe program is running and gets re-entered by another event, resulting in the loss of that span. As the concurrency increases, the likelihood of span loss escalates accordingly, thereby impacting the effectiveness of ZeroTracer. When comparing the count of missing probes in single-threaded and multi-threaded Bookinfo, the hooked eBPF programs in multi-threaded Bookinfo miss more probes (over 12%) compared to those in the single-threaded version (nearly 7.3%). As the number of concurrent threads increases, the likelihood of executing multiple probes simultaneously also rises, leading to a higher chance of missing generated spans or failing to propagate context information.

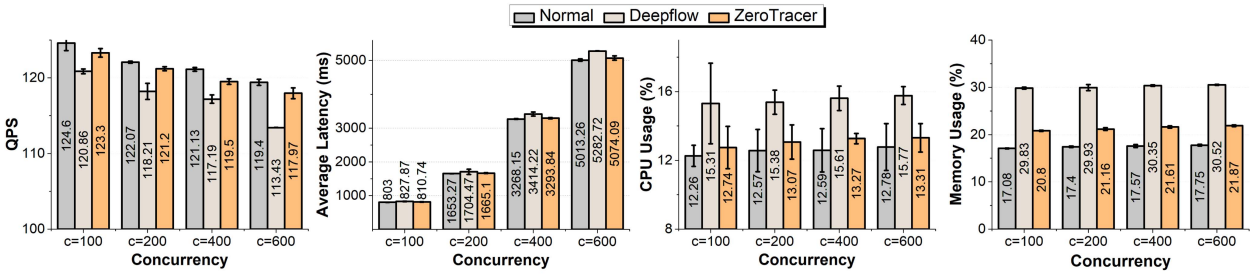
In non-invasive methods such as DeepFlow, the trace loss occurs due to incorrect correlation between spans. In such methods, all spans are captured, but their causal relationships are not well reconstructed, resulting in incomplete request traces. ZeroTracer does not suffer from such inaccuracies in request causality due to its precise causal propagation. Instead, the trace loss in ZeroTracer occurs when certain hooked functions are not triggered or are interrupted during execution, preventing the generation of corresponding spans. Consequently, the parent-child relationships among the remaining spans remain accurate, although some spans may be lost.

To ensure that ZeroTracer does not experience the span loss when exporting spans from eBPF programs to the agent via the ringbuffer, we compare the number of spans generated

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.



((a)) Comparison results on single-threaded Bookinfo.



((b)) Comparison results on multi-threaded Bookinfo.

Fig. 14. Overhead of ZeroTracer compared with Deepflow on service performance and resource consumption.

and exported by eBPF programs with the number received by the agent during the ten thousand concurrent workload testing under varying concurrency. We find that these counts are always consistent. To prevent buffer overflow, we set the ringbuffer size to 128 MB, which can hold up to 1,248 spans simultaneously. In our tests, ZeroTracer exports spans to the ringbuffer and retrieves them at the speed ranging from 10 MB/s to 35 MB/s. Therefore, any span loss in ZeroTracer is not caused by missing spans in the ringbuffer. The ring buffer size can be configured based on actual requirements.

E. (Q4) System Overhead

In this experiment, we assessed the overhead of ZeroTracer in tracking requests for the single-threaded and multi-threaded Bookinfo application compared to Deepflow. Workloads are generated with varying levels of concurrency: 100, 200, 400, and 600 concurrent users. The experiments run for ten minutes and are repeated five times on five physical machines. The average results are illustrated in Fig. 14.

First, we examine the impact of ZeroTracer on the service performance. As shown in Fig. 14, ZeroTracer increases the request latency for single-threaded Bookinfo by about 5–25 milliseconds(ms) (0.5% –0.9%) compared to the normal case, and reduces the QPS of the service by about 0.5–1.1 (0.4% –0.9%), even under 600 concurrent user requests. For multi-threaded Bookinfo, ZeroTracer increases service latency by 7–61 ms (0.7% –1.2%) and reduces QPS by 0.8–1.6 (0.7% –1.3%). The additional latency comes from the execution of our customized eBPF programs for intercepting and parsing the sending and receiving messages in the Linux kernel. In our implementation, the multi-threaded Bookinfo processes a request in 9.54 ms, which is faster than the single-threaded version at 10.83 ms when handling a single concurrent request. However, as concurrency increases, the multi-threaded Bookinfo requires more time to serve concurrent user requests than its single-threaded counterpart. This is because the multi-threaded version relies on asynchronous threads to handle user requests, which introduces thread waiting time. As concurrency rises, these waiting time, along with the scheduling overhead of the newly generated threads, further increase the execution delays.

Regarding resource consumption, ZeroTracer has increased CPU usage by 0.35% –0.51% (3% –5.8% relative to the normal) for single-threaded Bookinfo and 0.48% –0.68% (3.9% –5.4% relative to the normal) for multi-threaded Bookinfo, due to the execution of eBPF programs in the Linux kernel. The multi-threaded Bookinfo requires additional CPU resources for thread scheduling compared to the single-threaded version. In terms of memory usage, ZeroTracer needs additional 3.2% –3.7% (about 524MB–602 MB) memory for single-threaded Bookinfo to temporarily store partial message data, generated trace contexts, and other relevant information. For multi-threaded Bookinfo, the additional memory usage increases to 3.7% –4.2% (approximately 602MB–688 MB) to accommodate temporary data storage. The multi-threaded case demands more memory than the single-threaded case because it must also track parent-child relationships between threads.

In the meantime, Deepflow increases request latency and reduces QPS by 3.0% –5.8% for single-threaded Bookinfo and 3.0% –5.3% for multi-threaded Bookinfo. This overhead is higher than that of ZeroTracer, as Deepflow mounts more eBPF programs in the kernel. Additionally, it consumes 22% –37% more of CPU and 12.5% –14.6% more of memory resources, due to the execution of its server and the hooked eBPF programs.

To gain insights into the overhead imposed by our attached eBPF programs, we measured the average execution time required for each type of eBPF program mounted in the kernel.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

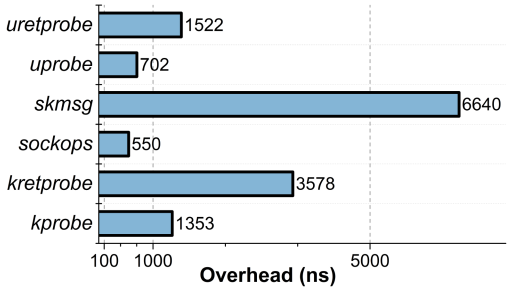


Fig. 15. Time overhead of the attached eBPF programs of ZeroTracer.

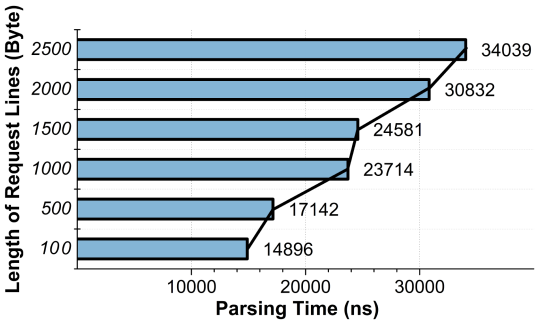


Fig. 16. Time overhead of ZeroTracer when parsing request lines with different lengths.

This measurement is conducted by repeatedly requesting the Bookinfo application 20000 times and invoking the hooked functions over 20000 times. The average latency results are depicted in Fig. 15. The additional latency introduced by eBPF programs of ZeroTracer ranges from 550 ns (nanoseconds) to 6640 ns. Notably, the `sk_msg` program exhibits the highest overhead at 6640 ns. This is attributed to its task of parsing outgoing messages and injecting trace contexts, which incurs significant processing time. It is important to note that eBPF imposes limitations on the number of instructions of an eBPF program, thereby limiting the latency of an eBPF program.

Since ZeroTracer employs the tail call mechanism of eBPF to parse messages with long request lines, we conduct tests to evaluate its performance under varying request line lengths. We generate 20,000 workloads with 100 concurrent users for Bookinfo to measure the time required for ZeroTracer to parse request lines of varying lengths. The results are shown in Fig. 16. As observed, ZeroTracer takes longer time to parse request lines as their length increases. On average, it processes a 100-byte request line in 14,896 ns, whereas a 2,500-byte request line takes 34,039 ns. The time overhead for parsing request lines increases almost linearly with length, as ZeroTracer requires more iterations to locate the end of the request line as its length grows.

Concluded above, ZeroTracer exhibits stable effects on service performance and resource consumption, regardless of the number of concurrent requests handled by the services. Among the attached eBPF programs, the `sk_msg` programs spends the most time parsing messages and injecting trace contexts in ZeroTracer. The parsing time increases as the request line length grows.

F. Deployment Experience: Demonstration in a Real-World Cloud Tracing Console

We have successfully deployed ZeroTracer in a real-world containerized environment and exported the collected data to a tracing console provided by the largest e-commerce company in China for visualization. A trace tree and its spans generated by ZeroTracer when a user requests Bookinfo is depicted in Fig. 17(a). Additionally, the trace is displayed in a tracing console in Fig. 17(b). When serving a request from a user, the Productpage service of Bookinfo calls the Detail service and the Review-v1 service, resulting in a total of 5 generated spans. This trace tree does help in troubleshooting faults. For instance, when the entry span 1 exhibits anomalous latency, tracking through the trace tree with span 2 and span 3 facilitates the identification of the abnormal Detail service.

G. Discussion

ZeroTracer implements an online non-invasive distributed tracing system for microservice applications serving the HTTP requests. But there exist some limitations that need to be discussed.

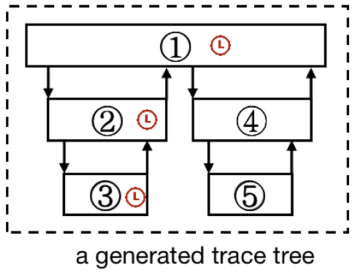
Multi-protocol Supports: When transmitting contexts between services, ZeroTracer embeds the context in the message header to maintain request causality. However, for certain encryption protocols where messages are encrypted before transmission, ZeroTracer intercepts only the encrypted packets rather than plaintext content. In such cases, determining the appropriate location for adding context becomes challenging, as ZeroTracer cannot effectively parse encrypted or unstructured requests. Incorrectly injecting context may disrupt the transmission and interpretation of the original packet. To address the challenges posed by encryption protocols, we plan to fix the context injection place at the beginning of the sending packets (offset 0) and deploy an additional eBPF program on the receiving service to extract and remove the context before packet processing, as shown in Fig. 18. This ensures that the original packet remains intact and can be correctly parsed by the receiving service. By doing so, ZeroTracer eliminates the need for message parsing during context injection, making it protocol-independent and applicable to various protocols, including encryption protocols, like gRPC and HTTPS. Alternatively, the context can be injected into the original message before encryption, allowing us to modify the plaintext message and embed the trace context before encryption. For example, the `Invoke` function in gRPC can provide insights into the message-sending procedure to help inject trace contexts.

Runtime Requirements: eBPF programs can only be deployed by privileged users. But Linux has the plan to make them available to unprivileged users [46], and there exists some work on unleashing unprivileged eBPF programs [47]. The requirement to root privilege of eBPF programs may be alleviated in the future. In addition, ZeroTracer requires a Linux kernel version of at least 5.2 [48] to execute.

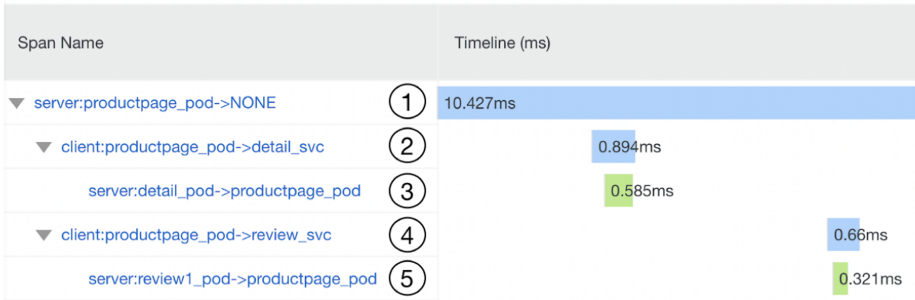
Function Updates: The mounting of eBPF programs to some user functions requires ZeroTracer to obtain the addresses of the user functions in different third-party libraries. To achieve

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

Span	Span ID	Parent ID
①	019e381400000000	0000000000000000
②	019e1e2a02000000	019e381400000000
③	019d019e03000000	019e1e2a02000000
④	019ec11d05000000	019e381400000000
⑤	022f019e01000000	019ec11d05000000



((a)) The trace tree generated by ZeroTracer.



((b)) Visualization of the trace tree in the console.

Fig. 17. Visualization in real-world tracing console of a generated trace of Bookinfo with its trace ID as *9e01f10abe23f10198f30e00104c5607*.

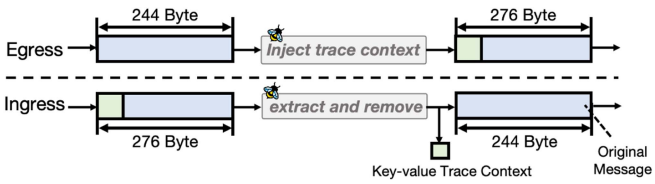


Fig. 18. Inter-service context propagation through a message using other protocols than HTTP.

accurate end-to-end request tracing, ZeroTracer must track user-space procedures such as coroutine scheduling, thread creation, and request connection handling (as shown in Table I). Unlike existing invasive distributed tracing methods that require source code modifications when third-party libraries or applications are updated, ZeroTracer only needs to update the addresses of the hooked user functions without altering the source code or simply add new eBPF programs when user-space procedures like coroutine scheduling, thread creation, and request connection handling update. The eBPF programs hooked by ZeroTracer on such user-space procedures are always decoupled from the applications.

Compatibility: For certain hardware or kernel bypass methods, such as DPDK, eBPF programs can be attached to user-space functions responsible for sending and receiving messages, similar to the socket channel example, to capture transmitted messages. This allows ZeroTracer to parse trace contexts from incoming requests and inject them into outgoing requests for inter-service context propagation. Even though some kernel bypass methods use dedicated hardware and libraries, it remains feasible to hook eBPF programs onto their message transmission functions, just as ZeroTracer does with socket-based message channels by hooking `sock_sendmsg` and `sock_recvmsg`. Additionally, by attaching to these user-space functions, ZeroTracer can track which thread processes each request, such as sending or receiving an HTTP request, enabling precise parent-child thread relationship tracking and facilitating intra-service context propagation. In addition, ZeroTracer is compatible with the existing OpenTelemetry and does not affect the request tracing of OpenTelemetry.

Multi-language Supports: ZeroTracer requires attaching eBPF programs to user functions to capture task scheduling and creation information, enabling context propagation through multi-threaded applications. For Golang and C++ programs, locating the addresses of hooked functions is straightforward. However, for Java and Python, which run on virtual machines, retrieving dynamic function addresses to attach eBPF programs is challenging. In such cases, ZeroTracer leverages bytecode instrumentation tools, such as javaagent [39], to hook user functions and extract task scheduling and creation details.

Extended Use Cases: ZeroTracer supports online key-value data injection into in-band requests and can be extended as a tool for service orchestration. For example, we can implement fine-grained grayscale release by injecting specific key-value tags into the end-to-end requests in practice.

VI. Related Work

eBPF-enabled observability: eBPF is widely used to enable fine-grained monitoring and improve system observability. Rolando Brondolin [49] proposed an eBPF-based non-transparent monitoring approach. Weave Scope [50] tracks TCP connection and improves the observability of Kubernetes clusters. bpftrace [51] helps trace in kernel. Taekho Nam et al. [52]

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

implement eBPF-based packet tracing on multiple network interfaces in Linux. Chang Liu et al. [53] collect user application L7/L4 layer network protocol interaction information based on eBPF. vNetTracer [54], [55] provides programmable packet tracing based on eBPF. Elastic [56] proposes a universal and continuous profiling tool using eBPF. Since eBPF enables efficient, real-time monitoring and tracing at the kernel level, providing deep system insights without altering application code, it makes trace generation in ZeroTracer feasible.

Packet tracing: Packet tracing capture packets between two services and helps generate traces for single-hop requests. Inkle [57] tracks single-hop gRPC requests for microservice applications on Kubernetes. tcpdump [58], [59] and WireShark [60] are able to sniff packets on physical links. Istio [61] allows to collect tracing data through its deployed proxies. INT-Label [62] introduces a lightweight telemetry architecture by periodically labeling packets. Pixie [63] enables tracking packets by monitoring sending and receiving functions with eBPF. The collected packets can help generate single-hop traces and serve as a data source for multi-hop trace generation using aggregation algorithms in existing distributed tracing systems.

Existing distributed tracing systems: Invasive tracing systems require developers to instrument tracing codes into application codes to generate traces. Zipkin [12], Jaeger [13], SkyWalking [17], OpenTracing [15], OpenTelemetry [14] are some typical examples of invasive tracing systems commonly used in practice. X-Trace [10] provides a comprehensive view of systems as a tracing framework. Pinpoint [9] tracks client requests in its communications layer. Dapper [11] implements a low-overhead distributed system by sampling. REPTrace [64] captures the complete end-to-end execution paths by utilizing the transferring messages. Pedro Las-Casas, et al. [65] and Hindsight [66] explore and optimize the sampling mechanism of distributed tracing systems. Castanheira, Lucas et al. [67] proposed a flexible distributed tracing workflow based on instrumented annotations and preemptive tracing for debugging. JCallGraph [18] realizes a distributed tracing system for microservice applications by instrumenting the cloud platform middlewares. Chunghan Lee et al. [68] enhance the single-hop request latency measurement with eBPF based on the invasive trace contexts in HTTP payloads. Existing non-invasive tracing systems generate traces without any instrumentation into applications and third-party libraries. Cha, Donghun, et al. [19] constructed the request traces based on the information collected by the service proxies in the service mesh. DeepFlow [20] mounts eBPF programs to collect commonly used metrics and generate traces by aggregation algorithms. OpenTelemetry [69] provides instrumentation for Go libraries using eBPF. Odigos [70] implements non-invasive tracing by integrating OpenTelemetry Golang automatic instrumentation [69] and Java third-party library instrumentation. However, these non-invasive approaches suffer from trace loss, dependency on third-party libraries, or do not support various high-level languages. Compared with the existing non-invasive approaches, ZeroTracer supports online non-invasive request tracing and can be adapted to different microservice implementations and various high-level languages, while maintaining high trace accuracy.

VII. Conclusion

In this paper, we propose ZeroTracer to achieve online end-to-end request tracing in a large-scale distributed system without any code instrumentation. Built on eBPF, ZeroTracer adapts to tracking requests for microservice application implementations across various high-level languages. Our comprehensive evaluations demonstrate that ZeroTracer can track over 91% of requests with reliable tracing performance, even when microservices handle requests in multi-threaded mode with increased concurrency. Furthermore, our experiments indicate that ZeroTracer has little impact on service performance, causing only a slight 0.5% –1.2% increase in service latency. For resource consumption, ZeroTracer consumes an additional 3% –5.8% of CPU and memory resources. This combination of effectiveness and efficiency positions ZeroTracer as a promising solution for tracking end-to-end requests in microservice systems.

VII. Data Availability

All the data and results of ZeroTracer can be found in the Github repository as follows: https://github.com/IntelligentDDS/zerotracer.

Acknowledgment

The authors greatly appreciate the insightful feedback from the anonymous reviewers. Special thanks are also extended to Guangba Yu (School of Computer Science and Engineering, Sun Yat-sen University; yugb5@mail.sysu.edu.cn), Zilong He (School of Computer Science and Engineering, Sun Yat-sen University; hezlong@mail2.sysu.edu.cn) for their useful comments on writing of this paper, and to Haibin Zhang (Alibaba Group; zhb192729@alibaba-inc.com) for his assistance in setting up the experimental environment. The corresponding author is Pengfei Chen.

References

[1] G. F. Coulouris, J. Dollimore, and T. Kindberg, *Distributed Systems: Concepts and Design*. London, U.K.: Pearson Education, 2005.
[2] S. Mullender, *Distributed Systems*. New York City, NY, USA: ACM, 1990.
[3] M. Van Steen and A. S. Tanenbaum, "A brief introduction to distributed systems," *Computing*, vol. 98, pp. 967–1009, 2016.
[4] J. Waldo, G. Wyant, A. Wollrath, and S. Kendall, "A note on distributed computing," in *Proc. Mobile Object Syst. Towards Program. Internet: 2nd Int. Workshop*, 2005, pp. 49–64.
[5] J. Thönes, "Microservices," *IEEE Softw.*, vol. 32, no. 1, pp. 116–116, Jan. 2015.
[6] G. Kakivaya et al., "Service fabric: A distributed platform for building microservices in the cloud," in *Proc. 13th EuroSys Conf.*, 2018, pp. 1–15.
[7] S. Luo et al., "An in-depth study of microservice call graph and runtime performance," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 3901–3914, Dec. 2022.
[8] K. Cheng et al., "ProScale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1294–1312, Apr. 2023.
[9] M. Chen, E. Kiciman, E. Fratkin, A. Fox, and E. Brewer, "Pinpoint: Problem determination in large, dynamic internet services," in *Proc. Int. Conf. Dependable Syst. Netw.*, 2002, pp. 595–604.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

[10] R. Fonseca, G. Porter, R. H. Katz, and S. Shenker, "X-trace: A pervasive network tracing framework," in *Proc. 4th USENIX Symp. Networked Syst. Des. Implementation*, 2007, Art. no. 20.

[11] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google Inc., Tech. Rep., 2010. [Online]. Available: https://research.google.com/archive/papers/dapper-2010-1.pdf

[12] Zipkin, "Zipkin is a distributed tracing system," 2023. [Online]. Available: https://zipkin.io/

[13] Jaeger, "Jaeger: Open source, end-to-end distributed tracing," 2023. [Online]. Available: https://www.jaegertracing.io/

[14] OpenTelemetry, "Opentelemetry: High-quality, ubiquitous, and portable telemetry to enable effective observability," 2023. [Online]. Available: https://opentelemetry.io/

[15] OpenTracing, "Opentracing: Vendor-neutral APIs and instrumentation for distributed tracing," 2023. [Online]. Available: https://opentracing.io/

[16] U. Erlingsson, M. Peinado, S. Peter, M. Budiu, and G. Mainar-Ruiz, "Fay: Extensible distributed tracing from kernels to clusters," *ACM Trans. Comput. Syst.*, vol. 30, no. 4, pp. 1–35, 2012.

[17] Apache, "Skywalking: Application performance monitor tool for distributed systems, especially designed for microservices, cloud native and container-based (kubernetes) architectures," 2023. https://skywalking.apache.org/

[18] H. Liu et al., "JCallGraph: Tracing microservices in very large scale container cloud platforms," in *Proc. Cloud Comput.: 12th Int. Conf., Held Part Serv. Conf. Federation*, San Diego, CA, USA, 2019, pp. 287–302.

[19] D. Cha and Y. Kim, "Service mesh based distributed tracing system," in *Proc. Int. Conf. Inf. Commun. Technol. Convergence*, 2021, pp. 1464–1466.

[20] J. Shen et al., "Network-centric distributed tracing with deepflow: Troubleshooting your microservices in zero code," in *Proc. ACM SIGCOMM 2023 Conf.*, 2023, pp. 420–437.

[21] J. Corbet, *Extending Extended BPF*. Linux Weekly News, 2014. [Online]. Available: https://lwn.net/Articles/603983/

[22] eBPF, "eBPF," 2023. [Online]. Available: https://ebpf.io

[23] J. Kazanavičius and D. Mažeika, "Evaluation of microservice communication while decomposing monoliths," *Comput. Inform.*, vol. 42, no. 1, pp. 1–36, 2023.

[24] N. Hamo and S. Saberian, "Evaluating the performance andusability of http vs grpc in communication between microservices," 2023. [Online]. Available: https://www.diva-portal.org/smash/get/diva2:1768795/FULLTEXT02.pdf

[25] OpenTelemetry, "Manual instrumentation for opentelemetry c," 2023. Accessed: Oct. 18, 2023. [Online]. Available: https://opentelemetry.io/docs/instrumentation/cpp/manual/

[26] S. McCanne and V. Jacobson, "The BSD packet filter: A new architecture for user-level packet capture," in *Proc. USENIX Winter Conf.*, 1993, Art. no. 2.

[27] Z. Ahmed, M. H. Alizai, and A. A. Syed, "Inkev: In-kernel distributed network virtualization for dcn," *SIGCOMM Comput. Commun. Rev.*, vol. 46, no. 3, pp. 1–6, 2018.

[28] S. Jouet and D. P. Pezaros, "BPFabric: Data plane programmability for software defined networks," in *Proc. ACM/IEEE Symp. Architectures Netw. Commun. Syst.*, 2017, pp. 38–48.

[29] S. Baidya, Y. Chen, and M. Levorato, "eBPF-based content and computation-aware communication for real-time edge computing," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2018, pp. 865–870.

[30] Facebook, "Katran: A high performance layer 4 load balancer," Accessed: Feb. 06, 2023. [Online]. Available: https://github.com/facebookincubator/katran

[31] M. Xhonneux, F. Duchene, and O. Bonaventure, "Leveraging eBPF for programmable network functions with ipv6 segment routing," in *Proc. 14th Int. Conf. Emerg. Netw. Exp. Technol.*, 2018, pp. 67–72.

[32] Y. Choe, J.-S. Shin, S. Lee, and J. Kim, ""eBPF/XDP based network traffic visualization and dos mitigation for intelligent service protection," in *Proc. Int. Conf. Emerg. Internetworking, Data Web Technol.*, 2020, pp. 458–468.

[33] S. Qi, L. Monis, Z. Zeng, I.-C. Wang, and K. K. Ramakrishnan, "SPRIGHT: Extracting the server from serverless computing! high-performance eBPF-based event-driven, shared-memory processing," in *Proc. ACM SIGCOMM 2022 Conf.*, 2022, pp. 780–794.

[34] S.-C. Oh and S.-W. Kim, "An efficient Linux kernel module supporting TCP/IP offload engine on grid," in *Proc. 5th Int. Conf. Grid Cooperative Comput.*, 2006, pp. 228–235.

[35] Bcc, "BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more," Accessed: Feb. 06, 2023. [Online]. Available: https://github.com/iovisor/bcc

[36] libbpf, "LIBBPF: A userspace library for loading and interacting with BPF programs," Accessed: Feb. 06, 2023. [Online]. Available: https://github.com/libbpf/libbpf

[37] D. Bernstein, "Containers and cloud: From LXC to docker to kubernetes," *IEEE Cloud Comput.*, vol. 1, no. 3, pp. 81–84, Sep. 2014.

[38] R. Rosen, "Resource management: Linux kernel namespaces and cgroups," *Haifux*, vol. 186, Art. no. 70, 2013.

[39] "Java agent," Accessed: Mar. 02, 2025. [Online]. Available: https://opentelemetry.io/docs/zero-code/java/agent/

[40] Fortio, "Fortio load testing library, command line tool, advanced echo server and web UI in GO (golang)," Accessed: Feb. 06, 2023. [Online]. Available:https://github.com/fortio/fortio

[41] Bookinfo, "Bookinfo," Accessed: Feb. 06, 2023. [Online]. Available: https://istio.io/latest/docs/examples/bookinfo/

[42] FudanSELab, "Train ticket - a benchmark microservice system," Accessed: Feb. 06, 2023. [Online]. Available: https://github.com/FudanSELab/train-ticket

[43] U. Saqib, "eBPF re-entrancy," in eBPF Submit, 2023. [Online]. Available: https://ebpf.io/summit-2023-talks/

[44] "Kprobes missing issue," Accessed: Mar. 28, 2025. [Online]. Available: https://github.com/iovisor/bcc/issues/4198

[45] Linux, "Kprobe implementations in linux," Accessed: Mar. 28, 2025. [Online]. Available: https://github.com/torvalds/linux/blob/master/kernel/kprobes.c#L1210

[46] J. Corbet, "Unprivileged BPF()," 2015. Accessed: Sep. 06, 2023. [Online]. Available: https://lwn.net/Articles/660331/

[47] S. Y. Lim, X. Han, and T. Pasquier, "Unleashing unprivileged eBPF potential with dynamic sandboxing," in *Proc. 1st Workshop eBPF Kernel Extensions*, New York, NY, USA, 2023, pp. 42–48. [Online]. Available: https://doi.org/10.1145/3609021.3609301

[48] Bcc, "Bpf features by Linux kernel version," 2023. Accessed: Sep. 06, 2023. [Online]. Available: https://github.com/iovisor/bcc/blob/master/docs/kernel-versions.md

[49] R. Brondolin and M. D. Santambrogio, "A black-box monitoring approach to measure microservices runtime performance," *ACM Trans. Archit. Code Optim.*, vol. 17, no. 4, pp. 1–26, Nov. 2020, doi: 10.1145/3418899.

[50] D. Calavera and L. Fontana, "Linux observability with BPF: Advanced programming for performance analysis and networking. o'reilly media," 2019. [Online]. Available: https://books.google.com.hk/books?id=jtS-DwAAQBAJ

[51] bpftrace, "BPFtrace: High-level tracing language for linux eBPF," Accessed: Mar. 01, 2023. [Online]. Available: https://github.com/iovisor/bpftrace

[52] T. Nam and J. Kim, "Open-source io visor eBPF-based packet tracing on multiple network interfaces of Linux boxes," in *Proc. Int. Conf. Inf. Commun. Technol. Convergence*, 2017, pp. 324–326.

[53] C. Liu, Z. Cai, B. Wang, Z. Tang, and J. Liu, "A protocol-independent container network observability analysis system based on eBPF," in *Proc. IEEE 26th Int. Conf. Parallel Distrib. Syst.*, 2020, pp. 697–702.

[54] K. Suo, Y. Zhao, W. Chen, and J. Rao, "Demo/poster abstract: Efficient and flexible packet tracing for virtualized networks using eBPF," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2018, pp. 1–2.

[55] K. Suo, Y. Zhao, W. Chen, and J. Rao, "vNetTracer: Efficient and programmable packet tracing in virtualized networks," in *Proc. IEEE 38th Int. Conf. Distrib. Comput. Syst.*, 2018, pp. 165–175.

[56] Elastic, "Drive optimization and sustainability with continuous profiling," Accessed: Mar. 02, 2025. [Online]. Available: https://www.elastic.co/observability/universal-profiling

[57] A. Perdanaputra and A. I. Kistijantoro, "Transparent tracing system on GRPC based microservice applications running on kubernetes," in *Proc. 7th Int. Conf. Adv. Inform.: Concepts, Theory Appl.*, 2020, pp. 1–5.

[58] tcpdump, "TCPDUMP & LIBPCAP," Accessed: Mar. 01, 2023. [Online]. Available: https://www.tcpdump.org/

[59] P. Goyal and A. Goyal, "Comparative study of two most popular packet sniffing tools-Tcpdump and Wireshark," in *Proc. 9th Int. Conf. Comput. Intell. Commun. Netw.*, 2017, pp. 77–81.

[60] WireShark, "The world's most popular network protocol analyzer," Accessed: Mar. 01, 2023. [Online]. Available: https://www.wireshark.org/

[61] Istio, "Istio: Simplify observability, traffic management, security, and policy with the leading service mesh," Accessed: Feb. 06, 2023. [Online]. Available: https://istio.io

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.

[62] E. Song et al., "INT-label: Lightweight in-band network-wide telemetry via distributed labeling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 5, pp. 751–767, May 2024.
[63] Pixie, "Pixie: Open source kubernetes observability for developers," Accessed: Mar. 02, 2025. [Online]. Available: https://px.dev
[64] Y. Yang, L. Wang, J. Gu, and Y. Li, "Capturing request execution path for understanding service behavior and detecting anomalies without code instrumentation," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 996–1010, Mar./Apr. 2023.
[65] P. Las-Casas, J. Mace, D. Guedes, and R. Fonseca, "Weighted sampling of execution traces: Capturing more needles and less hay," in *Proc. ACM Symp. Cloud Comput.*, 2018, pp. 326–332.
[66] L. Zhang, Z. Xie, V. Anand, Y. Vigfusson, and J. Mace, "The benefit of hindsight: Tracing edge-cases in distributed systems," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation*, 2023, pp. 321–339.
[67] L. Castanheira, T. A. Benson, and A. Schaeffer-Filho, "The case for more flexible distributed tracing," in *Proc. Student Workshop*, 2020, pp. 27–28.
[68] C. Lee, R. Yoshitani, and T. Hirotsu, "Enhancing packet tracing of microservices in container overlay networks using eBPF," in *Proc. 17th Asian Internet Eng. Conf.*, New York, NY, USA, 2022, pp. 53–61, doi: 10.1145/3570748.3570756.
[69] OpenTelemetry, "Opentelemetry auto instrumentation using eBPF," 2023. Accessed: Oct. 18, 2023. [Online]. Available: https://github.com/open-telemetry/opentelemetry-go-instrumentation/
[70] Odigos, "ODIGOS: Generate distributed traces for any application without code changes," Accessed: Mar. 02, 2025. [Online]. Available: https://github.com/odigos-io/odigos



Wanqi Yang received the BS degree in computer science and technology from Sun Yat-Sen University, Guangzhou, China. She is currently working toward the PhD degree in computer science and technology with the School of Computer Science and Engineering, Sun Yat-Sen University. Her research interests include eBPF, cloud computing, and computer network.



Pengfei Chen received the graduation degree from the Department of Computer Science, Xi'an Jiaotong University and the PhD degree, in 2016. He is currently a professor in the School of Computer Science and Engineering of Sun Yat-sen University. Meanwhile, he is a PhD advisor. He is interested in distributed systems, AIOps, cloud computing, Microservice and BlockChain. Especially, he has strong skills in cloud computing. So far, Dr. Chen has published more than 60 papers in some international conferences including ACM/IEEE ASE, ACM/IEEE ICSE, IEEE INFOCOM, WWW and journals including *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Neural Networks and Learning Systems*, *IEEE Transactions on Reliability*, *IEEE Transactions on Services Computing*. He serves as of program committee member of multiple conferences and reviewers of some internal journals such as *IEEE Transactions on Parallel and Distributed Systems*, *ACM Transactions on Software Engineering and Methodology*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Computers*.



Kai Liu received the master's degree in electronic and communication engineering from the Beijing University of Posts and Telecommunications, in 2020. He is currently a senior software engineer with Alibaba Cloud. His research interests encompass cloud-native architectures, microservices, and observability.



Huxing Zhang received the master's degree from Shanghai Jiaotong University. He is currently a staff engineer with Alibaba Cloud. His research interests include cloud native, microservices, and observability.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:08 UTC from IEEE Xplore. Restrictions apply.